

PREFACE

General Purpose Hyper-Resilient Edge Supercomputing.

"In the abundance of water, the fool is thirsty." – Robert Nesta Marley, circa 1976

Marabunta started as a charitable High-Performance Computing (HPC) engine meant to give Médecins Sans Frontières (Doctors Without Borders) and other non-governmental organizations the supercomputing power required to predict Ebola outbreaks and manage responses to natural or man-made catastrophes, utilizing donor-provided computational FLOPs and algorithms.

Distributing massive AI and scientific workloads into hostile, infrastructure-poor environments required solving the hardest fundamental problems in distributed systems: bandwidth saturation, state synchronization, and execution trust.

Cambrian Radiation subsequently realized Marabunta's broader applications. Having solved those primitives to ensure the survivability of the network, a generic architecture naturally resulted from those emerging traits.

The system evolved into a substrate for executing any deterministic code, anywhere on Earth, with cryptographic proof of accuracy and physical location. Thus, it seems to have become commercially viable.

MARABUNTA REDONE

PREFACE: ASYMMETRIC DISTRIBUTED COMPUTE

General Purpose Hyper-Resilient Edge Supercomputing.

The Hyperscaler Paradigm is physically constrained by **Data Gravity** and **Egress Economics**. When processing massive datasets at the point of origin—or when executing logic within restricted, air-gapped jurisdictions—transmitting data to a centralized cluster violates the fundamental laws of network physics and sovereign economics.

Marabunta is an asymmetric execution substrate. It provides a sovereign alternative for environments where the data must remain at the origin.

The Marabunta Swarm is a decentralized mesh designed to coordinate arbitrary heterogeneous topologies. It inhabit high-density enterprise clusters, air-gapped corporate intranets, and consumer silicon alike, harnessing ambient thermodynamic dissipation to execute massive, deterministic workloads.

This document details the mechanical implementation of the core pillars required for sovereign edge compute: * **Physical Identity**: Deriving cryptographic proof of existence from CPU thermodynamic expenditure. * **Deterministic Sandboxing**: Executing untrusted code within strict memory and fuel boundaries. * **Segmented Routing**: Maintaining network consensus across firewalled and high-latency topologies.

This is the technical specification of a production-ready distributed engine.

```
# Illustrative: Monitoring local node integration with the global mesh
$ mrb swarm status --comprehensive
```

VOLUME 1: THE THERMODYNAMIC FOUNDATION

1.1 Identity in a Zero-Trust Vacuum (The Chrysalis Grinder)

Centralized Identity Management (IAM) is an architectural bottleneck in decentralized networks. Marabunta replaces the master server with a hardware-anchored identity system.

To participate in the Swarm, a node must bleed computational energy. During the initial boot phase, the node executes the **Chrysalis Grinder**, a memory-hard Proof-of-Work (PoW) algorithm. By saturating the L3 cache and the main memory bus using a unique hardware seed and core identifier, the node derives its sovereign **Nodeld**.

Let E be the minimum thermodynamic energy expenditure required to discover a valid cryptographic nonce n , where D is the target network difficulty, L_{mem} is the latency of the main memory bus, and μ is the cache miss rate. The fundamental work equation is bounded by:

$$E \geq \left(\frac{2^{256}}{D} \right) \times \left(P_{idle} \cdot t_{hash} + P_{active} \cdot \mu \cdot L_{mem} \right)$$

This process mathematically neuters the efficiency of custom ASICs, which inherently possess low L_{mem} but suffer a massive μ penalty under this specific memory-hard algorithm. A Marabunta identity is a cryptographic receipt of physical work.

```
# Illustrative: Commencing hardware-anchored identity derivation
$ mrb identity derive --difficulty 0x000FFFF...
```

1.2 The Thermal Guillotine

Because Marabunta can execute untrusted code on consumer-grade hardware, it must assume environments where thermal dissipation is unmanaged. To prevent malicious algorithms or runaway simulations from causing physical damage to host silicon, the system implements the **Thermal Guillotine**.

The orchestrator continuously monitors physical CPU temperatures. Let $T_{core}(t)$ represent the internal silicon temperature at time t , and T_{crit} represent the absolute thermal ceiling (e.g., $91^{\circ}C$). The protection mechanism executes when the following threshold is breached:

$$\int_{t_0}^{t_1} \frac{dT_{core}}{dt} dt \geq T_{crit} \implies Interrupt(Epoch_{WASM})$$

If the threshold is breached—or if sensors are masked by a hypervisor—the daemon executes a hard interruption. By advancing the execution epoch of the underlying WebAssembly engine, the orchestrator terminates the process from outside its own execution boundary.

```
// Illustrative: Hardware health report during a sustained load event
GET /api/v1/node/thermal
{
  "state": "WARNING",
  "temperature": 88.2,
  "throttle_applied": true
}
```

1.3 The PARANOID Micro-Audit

High-priority tasks require consistent memory latency. To filter out weak hardware or virtualized environments that swap memory to disk, the orchestrator enforces a **Micro-Audit** (verify_mode: PARANOID).

The orchestrator demands the node allocate a 256MB matrix and execute a deterministic transformation within $2,500ms$. We utilize `black_box` compiler directives to ensure the OS cannot optimize the allocation away. Let τ_{alloc} be the time to allocate and $\tau_{transform}$ be the time to complete the deterministic hash matrix H_{matrix} . The condition for network acceptance is:

$$\tau_{alloc} + \tau_{transform}(H_{matrix}) \leq 2500ms$$

If the node fails this latency test, it is rejected from the high-performance queue. Only bare-metal hardware with high-speed physical RAM is permitted to claim sensitive payloads.

VOLUME 2: THE TOPOLOGY OF CHAOS

2.1 Statistical Kademlia Density Estimation

Marabunta strictly caps its local Kademlia representation to $k = 10,000$ peers to preserve host RAM. To accurately monitor a planetary-scale network without catastrophic memory exhaustion, the daemon utilizes **Statistical Kademlia Density Estimation**.

By measuring the mathematical XOR distance to its closest neighbors, the node probabilistically extrapolates the global topology. If b_{max} is the index of the deepest populated k-bucket (representing the longest shared binary prefix), the estimated global network size $\hat{N}$ is derived as:

$$\hat{N} = 2^{b_{max}} \times \left(\frac{k}{p_{collision}} \right)$$

The orchestrator maintains $O(\log N)$ routing efficiency and accurate global telemetry without holding the entire network state in memory.

```
// Illustrative: Fetching extrapolated Swarm topology
GET /api/v1/swarm/density
{
  "known_peers": 10000,
  "estimated_global_scale": 12500400,
  "confidence_interval": 0.95
}
```

2.2 Thermodynamic Gossip Jitter

Epidemic Gossip protocols suffer from “thundering herd” bottlenecks. To maintain thermodynamic equilibrium, Marabunta implements **Adaptive Gossip Jitter**.

The background daemon dynamically evaluates the extrapolated node count $\hat{N}$. As the network scales, the base heartbeat interval Δt aggressively decelerates, and a randomized exponential backoff J is injected.

$$\Delta t = \tau_{base} \times \lceil \log_{10}(\hat{N}) \rceil + J(0, \tau_{base})$$

This mathematically guarantees that the global packet rate remains bounded and asymptotic, preventing the 460-million packet/sec meltdown that would otherwise DDoS residential ISP backbones.

2.3 Instantaneous Work Resolution ($O(1)$ Data Gravity)

A traditional orchestrator scanning a million pending tasks performs an $O(N)$ linear search, pinning the CPU at 100% just looking for work. Marabunta completely decouples task tracking from execution loops.

The `KnowledgeStore` maintains a strict $O(1)$ indexed queue of unassigned chunks. When a 64-core enterprise node requests work, the CPU instantly pops contiguous task fragments off the stack in constant time. The CPU spends zero cycles scanning arrays, unlocking 100% of the silicon for the WebAssembly sandbox.